

¿Qué es Flexbox en CSS?

Flexbox (cuyo nombre completo es Flexible Box Layout Module) es un modelo de diseño unidimensional de CSS. Su objetivo principal es proporcionar una forma más eficiente de diseñar, alinear y distribuir el espacio entre los elementos de un contenedor, incluso cuando sus tamaños son dinámicos o desconocidos.

Es la herramienta moderna y fundamental de CSS para controlar la disposición de los elementos en una **única dirección** (ya sea en fila o en columna).

Conceptos Clave

1. Modelo Unidimensional

Flexbox es *unidimensional*. Esto significa que organiza los elementos a lo largo de un **solo eje** a la vez:

- **Horizontal** (fila, por defecto)
- **Vertical** (columna)

Si necesitas controlar elementos en dos dimensiones (filas y columnas simultáneamente), la herramienta adecuada es **CSS Grid**.

2. Contenedor y Elementos

Flexbox se basa en una relación padre-hijo:

- **Contenedor Flexible (Flex Container):** Es el elemento padre al que se aplica la propiedad **display: flex;**. Esta es la clave que activa el modelo.
- **Elementos Flexibles (Flex Items):** Son los **hijos directos del contenedor**. Son estos elementos los que se alinean y distribuyen automáticamente según las reglas que apliques al contenedor.

3. Alineación Sencilla

Flexbox simplifica drásticamente tareas que históricamente eran difíciles o requerían *hacks* complejos en CSS, como:

- **Centrar un elemento:** Lograr el centrado horizontal y vertical con solo dos propiedades: `justify-content: center;` y `align-items: center;`.
- **Distribuir el espacio:** Repartir automáticamente el espacio libre entre los elementos (`space-between`, `space-around`).
- **Reordenar:** Cambiar el orden visual de los elementos (`order`) sin modificar el HTML.

Conceptos Fundamentales de Flexbox

1. El Contenedor Flexible (Flex Container)

Este es el elemento padre al que aplicas `display: flex;`. Al hacer esto, todos sus hijos directos se convierten en Elementos Flexibles (Flex Items).

- **Propiedad Clave:** `display: flex;`

2. Ejes de Flexbox

Flexbox funciona en torno a dos ejes principales:

- **Eje Principal (Main Axis):** La dirección a lo largo de la cual se colocan los elementos flexibles (por defecto, horizontal/fila).
 - o Propiedad para cambiar la dirección: `flex-direction` (ej: `row`, `column`).
- **Eje Cruzado (Cross Axis):** El eje perpendicular al principal (por defecto, vertical/columna).

3. Propiedades de Alineación (En el Contenedor)

Estas propiedades se aplican al **Contenedor** y controlan cómo se posicionan los **Ítems** dentro de él.

Propiedad	Eje que controla	Lo que hace	Valores Clave
<code>justify-content</code>	Principal (Horizontal)	Distribuye y alinea los ítems a lo largo de la línea.	<code>center</code> , <code>flex-start</code> , <code>flex-end</code> , <code>space-between</code> , <code>space-around</code>
<code>align-items</code>	Cruzado (Vertical)	Alinea los ítems a lo largo del eje perpendicular.	<code>center</code> , <code>flex-start</code> , <code>flex-end</code> , <code>stretch</code>

4. Propiedades de los Elementos Hijos (En el Ítem)

Estas propiedades se aplican directamente a los **Elementos Flexibles**.

Propiedad	Lo que hace	Propiedades relacionadas
order	Permite cambiar la posición visual de un elemento, independientemente de su orden en el HTML.	Los valores más bajos van primero. Por defecto es 0 .
flex-grow	Define la capacidad de crecimiento de un ítem para ocupar el espacio sobrante del contenedor.	Un valor de 1 o más permite que el ítem se expanda.
flex	Una propiedad atajo que combina flex-grow , flex-shrink y flex-basis (tamaño inicial).	Es la forma más común de controlar el tamaño flexible.

Práctica Guiada de Flexbox en 60 minutos

1. Configuración Inicial

Crea dos archivos: `index.html` y `styles.css`.

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Práctica Flexbox</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="contenedor">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
  </div>
</body>
</html>
```

styles.css (Estilos base)

```
.contenedor {
  border: 3px solid #3498db;
  background-color: #ecf0f1;
  height: 300px; /* Para ver la alineación vertical */
}

.item {
  background-color: #e74c3c;
  color: white;
  padding: 20px;
  margin: 5px;
```

```
    font-size: 1.5em;
    text-align: center;
}
```

2. Propiedades del Contenedor (.contenedor)

Ahora, modificaremos el selector **.contenedor** en **styles.css** para aplicar Flexbox y manipular los elementos.

1. Activar Flexbox y Dirección

Añade lo siguiente al **.contenedor**:

```
CSS
.contenedor {
    /* ... estilos base ... */
    display: flex;

    /* Dirección: Puedes cambiar el eje principal (main axis)
*/
    /* El valor por defecto es row. Prueba a cambiarlo: */
    /* flex-direction: column; */
}
```

Comenta `flex-direction: column;` para la siguiente parte.

2. Alineación en el Eje Principal (Horizontal por defecto)

Usa **justify-content** para distribuir los elementos a lo largo de la línea:

```
CSS
.contenedor {
    /* ... estilos anteriores ... */
    justify-content: flex-start; /* Por defecto: inicio */
    /* Prueba: flex-end, center, space-between, space-around,
space-evenly */
}
```

Contesta: Prueba los 5 posibles valores de justify-content y describe la diferencia entre space-between, space-around y space-evenly.

Respuesta: space-between pone los bloques de espacio a los lados, space-around los espacios rodead el div, y space-evenly hace que los espacios sean iguales

3. Alineación en el Eje Cruzado (Vertical por defecto)

Usa **align-items** para alinear los elementos perpendicularmente al eje principal:

CSS

```
.contenedor {  
  /* ... estilos anteriores ... */  
  /* Solo funciona porque el contenedor tiene un 'height'  
  definido */  
  align-items: stretch; /* Por defecto: estira los ítems */  
  /* Prueba: flex-start, flex-end, center, baseline */  
}
```

Contesta: ¿Cómo se pueden centrar los elementos **perfectamente** en el contenedor tanto horizontal como verticalmente?

Respuesta: justify-content y align-content

Propiedades de los Elementos Hijos (.item)

Modificaremos el selector **.item** en **styles.css**.

1. Orden

Usa **order** para cambiar la posición de un ítem, independientemente de su orden en el HTML. Los valores más bajos van primero (por defecto es 0).

CSS

```
/* Añade un estilo específico para el ítem 2 */  
.contenedor :nth-child(2) {  
  order: -1; /* Mueve el ítem 2 al principio */  
  background-color: #2ecc71;  
}
```

```
}  
  
.contenedor :nth-child(3) {  
  order: 1; /* Mueve el ítem 3 al final */  
  background-color: #9b59b6;  
}
```

Actividad: Intenta ordenar los ítems de mayor a menor.

2. Crecimiento (Growth) y Encogimiento (Shrink)

Estos controlan cómo los elementos consumen el espacio disponible o se adaptan si hay poco espacio.

- **flex-grow:** Factor de crecimiento. Por defecto es 0 (no crece).
- **flex-shrink:** Factor de encogimiento. Por defecto es 1 (puede encogerse).
- **flex-basis:** Tamaño inicial del ítem antes de que crezca o encoja (como width o height).

CSS

```
.contenedor :nth-child(1) {  
  /* ... */  
  flex-grow: 1; /* Crece una parte */  
  background-color: #3498db;  
}  
  
.contenedor :nth-child(2) {  
  /* ... */  
  flex-grow: 2; /* Crece el doble que el 1 */  
}  
  
/* El ítem 3 (sin flex-grow) mantiene su tamaño base */
```

Atajo: La propiedad **flex** es un atajo de flex-grow, flex-shrink y flex-basis.

flex: <flex-grow> <flex-shrink> <flex-basis>;

Ejemplo ----> flex: 1 0 200px;

3. Unidades de medidas relativas

Antes de realizar el desafío final, vamos a recordar una unidad de medida relativa muy utilizada para crear *layouts* que se adapten perfectamente a cualquier dispositivo, lo cual se integra muy bien con Flexbox.

Unidad	Significado	Relativo a...
vh	Viewport Height	El Ancho de la ventana de visualización (1vw = 1% del ancho).
vw	Viewport Width	El Ancho de la ventana de visualización (1vw = 1% del ancho).
vmin	Viewport Minimum	El menor valor entre vh y vw.
vmax	Viewport Maximum	El mayor valor entre vh y vw.

Así, para lograr que un elemento o sección ocupe **toda la altura visible de la pantalla** la primera vez que se carga la página (un diseño conocido como "pantalla completa" o *full-screen section*), haríamos algo como:

```
.seccion-principal {  
  
    height: 100vh; /* La altura de esta sección siempre será igual a la altura  
total de la pantalla */  
  
    width: 100vw; /* Junto con 100vw (Viewport Width), cubre toda la pantalla  
*/  
}
```

Desafío Final

Desafío: Creación de un Layout de Página Simple

Crea un nuevo contenedor (o borra el contenido del actual) para simular un *layout* típico de web: **Cabecera, Contenido Principal, Barra Lateral y Pie de Página**.

1. Estructura HTML:

```
<div class="layout">  
    <header class="bloque">Cabecera</header>
```



```
<main class="bloque">Contenido Principal</main>
<aside class="bloque">Barra Anuncios</aside>
<footer class="bloque">Pie de Página</footer>
</div>
```

2. Estilos CSS:

Contenedor:

- Coloca los elementos uno debajo del otro.
- Haz que los elementos ocupen toda la **altura** de la vista. (Necesario para que los espacios hijos puedan crecer)

Hijos:

- Dale espaciado 20px, borde sólido y alineación centrada a los hijos.
- Haz que el contenido **principal** ocupe todo el espacio sobrante. Resáltalo con un color de fondo amarillo claro.

3. Añade HTML

- Sustituye el contenido del main por el siguiente contenido:

```
<section>Sección principal</section>
<section>Sección secundaria</section>
```

4. Estilos CSS (El Reto):

- Coloca los elementos uno al lado del otro, con fondo verde esmeralda oscuro y margen de 5px.
- La sección secundaria debe estar primero y ocupar 200px de base. La sección primaria ocupará el resto.